T2 - Object-Oriented Python: classes, validation, inheritance, manager classes

Covers writing a class (`__init__`, `self`, methods, `__str__`), class vs instance attributes, validation that raises `ValueError`, inheritance with `super()`, lists of objects, and a "manager" class that holds a list. Earns the class/design and style marks in any task that uses OOP (and `class SimpleNN(nn.Module)` in the 2024 PyTorch task).

1. What the exam asks

OOP is not a headline task in the 2022-2024 Python papers, but it appears in three ways:

Where	Exact wording	What you need
2024 Python Task 3 [4+2+3+3]	"Define a class SimpleNN that extends torch.nn.Module."	a subclass with `super().__init__()` (template 3.7)
2024 Java Task 1 (same ideas, Java syntax)	"Name: ... must be at least 3 characters long." "Ensure that all the values that are stored are valid ... If someone tries to create or update an animal with invalid values, then an IllegalArgumentException should be thrown, and the change should not take effect."	validation raising `ValueError` (3.3, section 4)
2024 Java header	"marks will be awarded for clean code and sensible object-oriented design choices as well as functionality."	tidy classes, one job each
2024 Java Task 3	"PetService ... stores a single piece of information: the collection of enclosures ... initially empty."	a manager class holding a list (3.6)
Lab 01 (OOP)	"Create a Manager class that inherits from Employee ... use isinstance()" ; "eligible_for_training() ... return True or False ... rather than printing"	3.1, 3.4
tAPP-1 Tasks 5-8	"A strong solution keeps eligibility logic inside a method and keeps input/output or reporting orchestration outside the class."	methods RETURN, the loop outside PRINTS

NOTE: A Python-only exam can still say "use a class" for any task (a `Student`, `Book`, `Sale`). The 2022/2023 tkinter tasks were usually written as classes too. If the brief does not demand a class, a function is fine; if it does, copy the templates below.

TIP: Style marks look for: class names in `CamelCase`, attributes in `snake_case`, a one-line comment/docstring per class, methods that do ONE thing, and no `print` inside methods that should return a value.

2. Mental model (analogy)

- **Class** = a blank form (or cookie cutter). It says what fields exist, but holds no real data. **Object / instance** = one filled-in form (one cookie). `Employee("Som", 80)` fills in a new form. This is the course slide's own picture: "a class as an empty form".
- `__init__` = the person at the desk who fills in the new form the moment it is handed out. `self` = the name badge on the form: "this particular form". `self.name` = "the name field on THIS form".
- **Instance attribute** = your own desk drawer (`self.name`: different for each object). **Class attribute** = the notice board on the office wall (`company = "ABC Ltd"`): everyone sees the same one.
- **Method** = something the form can DO ("introduce yourself", "tell me if you qualify").
- `__str__` = the form's name tag text, what `print(obj)` shows. Without it you get `<__main__.Employee object at 0x...>`.
- **Property/setter with validation** = a bouncer at the door of each field. Bad value? `raise ValueError`, the field keeps its old content.
- **Inheritance** = job titles. A Manager IS-A Employee: it gets everything an Employee has, plus its own extras. `super()` = "first do what my parent does".
- **Manager class holding a list** = a librarian. The librarian (`Team`, `PetService`) keeps the shelf (list) of books (objects) and offers add / find / remove / report.

3. Copy-paste templates

3.1 A basic class (Lab 01 `instance.py`, tAPP `t6_oop-dec.py`)

```
class Employee:
    """A person who may be eligible for training."""
    company = "ABC Ltd"                      # class attribute (shared)

    def __init__(self, name, score, active=True):
        self.name = name                     # instance attributes
        self.score = score
        self.active = active

    def report(self):                        # a method: does something
        print(f"{self.name} : {self.score}")

    def eligible_for_training(self):         # returns a bool, never prints
        return self.active and self.score >= 70

    def __str__(self):                       # used by print(obj)
        return f"{self.name} (score {self.score})"

    def __repr__(self):                      # used inside lists / debugging
        return f"Employee({self.name!r}, {self.score!r}, {self.active!r})"

emp1 = Employee("Som", 80)
emp2 = Employee("Nidhi", 100, False)
emp1.report()                                # Som : 80
print(emp2)                                  # Nidhi (score 100)
print([emp1, emp2])
print(emp1.eligible_for_training(), emp2.eligible_for_training())  # T F
Employee.report(emp1)             # same as emp1.report(): self IS emp1
```

3.2 Class attribute vs instance attribute (Lab 01 `class_instance_attributes.py`)

```
print(emp1.company, emp2.company)            # ABC Ltd ABC Ltd
Employee.company = "XYZ plc"                 # via the CLASS: all see it
print(emp1.company, emp2.company)            # XYZ plc XYZ plc
emp1.company = "Solo Ltd"                    # via an object: only emp1 gets
print(emp1.company, emp2.company)            # its own copy: Solo Ltd XYZ plc

class Ticket:
    issued = 0                               # shared counter
    def __init__(self, owner):
        self.owner = owner
        Ticket.issued += 1                   # change the SHARED one
        self.number = Ticket.issued          # each ticket keeps its own

t1, t2 = Ticket("Amy"), Ticket("Ben")
print(t1.number, t2.number, Ticket.issued)   # 1 2 2
```

3.3 Validation: raise `ValueError` in `__init__` and in setters

Use when the brief says "must be valid" / "an exception should be thrown and the change should not take effect".

```
class Employee:                              # 3.1 class, now with checks
    def __init__(self, name, score, active=True):
        self.name = name                     # these lines call the setters
        self.score = score
        self.active = active

    @property
    def name(self):                          # reading: emp.name
        return self._name

    @name.setter
    def name(self, value):                   # writing: emp.name = "x"
        if not isinstance(value, str) or len(value.strip()) < 3:
```

```
            raise ValueError("Name must be at least 3 characters.")
        self._name = value.strip()            # only reached if valid

    @property
    def score(self):
        return self._score

    @score.setter
    def score(self, value):
        if not isinstance(value, (int, float)) or not 0 <= value <= 100:
            raise ValueError(f"Score must be 0–100, not {value!r}.")
        self._score = value

    def report(self):
        print(f"{self.name} : {self.score}")

e = Employee("Som", 80)
try:
    e.score = 150                             # rejected...
except ValueError as error:
    print("Rejected:", error)
print(e.score)                                # ...and still 80: no change
try:
    Employee("Al", 50)                        # name too short
except ValueError as error:
    print("Rejected:", error)
```

Simpler version with no `@property` (fine for marks, and what the course slides cover): validate in `__init__` and in a `set_score()` method, both calling the same check.

```
class Student:
    def __init__(self, name, mark):
        self.name = name
        self.mark = 0
        self.set_mark(mark)                   # reuse the one checker

    def set_mark(self, mark):
        if not 0 <= mark <= 100:              # check BEFORE assigning
            raise ValueError("Mark must be 0–100.")
        self.mark = mark

s = Student("Amy", 55)
try:
    s.set_mark(-5)
except ValueError as error:
    print(error, "Mark is", s.mark)       # Mark must be 0–100. Mark is 55
```

3.4 Inheritance, `super()`, overriding, `isinstance` (Lab 01 `inheritance.py`, tAPP `t8_team.py`)

```
class Employee:                               # parent (kept short here)
    def __init__(self, name, score, active=True):
        self.name = name
        self.score = score
        self.active = active
    def report(self):
        print(f"{self.name} : {self.score}")

class Manager(Employee):                      # child: Manager IS-A Employee
    def __init__(self, name, score, active=True, team=None):
        super().__init__(name, score, active) # parent sets name/score/active
        self.team = team if team is not None else []   # extra attribute

    def lead_team(self):                      # new method
        print(f"{self.name} is leading {len(self.team)} people")

    def report(self):                         # OVERRIDE the parent's method
        super().report()                      # reuse it, then add more
        print("   (manager)")

boss = Manager("Zhong", 80, True, ["Som", "Adi"])
worker = Employee("Nidhi", 100)
```

```
for person in [boss, worker]:                # same call, different behaviour
    person.report()
boss.lead_team()
print(isinstance(boss, Employee), isinstance(boss, Manager))      # True True
print(isinstance(worker, Manager))                                # False
```

3.5 A list of objects: loop, filter, search, total, max, sort

```
people = [Employee("Zhong", 80), Employee("Nidhi", 100),
          Employee("Som", 71, False), Employee("Adi", 69)]
for p in people:                             # loop OUTSIDE the class prints
    status = "eligible" if p.eligible_for_training() else "not eligible"
    print(f"{p.name}: {status}")
passed = [p for p in people if p.score >= 70]          # filter -> new list
print([p.name for p in passed])           # ['Zhong', 'Nidhi', 'Som']
average = sum(p.score for p in people) / len(people)   # total / count
best = max(people, key=lambda p: p.score)              # object with top score
by_score = sorted(people, key=lambda p: p.score, reverse=True)
print(f"{average:.1f}", best.name, [p.name for p in by_score])
found = None                                 # search by hand
for p in people:
    if p.name == "Adi":
        found = p
        break
print(found)                                 # uses __repr__ if no __str__
```

3.6 A "manager" class that holds a list (PetService / Team / Library)

```
class Team:
    """Holds a collection of Employee objects."""
    def __init__(self):
        self.members = []                    # starts EMPTY (2024 Task 3a)

    def add(self, employee):
        if not isinstance(employee, Employee):
            raise ValueError("Only Employee objects can join a Team.")
        self.members.append(employee)

    def find(self, name):                    # returns the object or None
        for member in self.members:
            if member.name == name:
                return member
        return None

    def remove(self, name):
        member = self.find(name)
        if member is None:                   # mirrors the Java exception
            raise ValueError(f"No member called {name}.")
        self.members.remove(member)

    def eligible(self):                      # RETURNS a list, no printing
        return [m for m in self.members if m.eligible_for_training()]

    def print_all(self):
        for member in self.members:
            print(member)

    def __len__(self):                       # makes len(team) work
        return len(self.members)

team = Team()
team.add(Employee("Zhong", 80))
team.add(Employee("Som", 71, False))
team.add(Employee("Adi", 69))
team.print_all()
print(len(team), [m.name for m in team.eligible()])      # 3 ['Zhong']
team.remove("Adi")
try:
    team.remove("Nobody")
```

```
except ValueError as error:
    print(error)                              # No member called Nobody.
```

3.7 Subclassing a library class (2024 Task 3: `SimpleNN(nn.Module)`)

```
import torch
import torch.nn as nn

class SimpleNN(nn.Module):                    # extends nn.Module
    def __init__(self):
        super().__init__()                    # FIRST line: sets up nn.Module
        self.fc1 = nn.Linear(10, 5)           # layers = attributes on self
        self.fc2 = nn.Linear(5, 3)
        self.out = nn.Linear(3, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):                     # how data flows through
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        return self.sigmoid(self.out(x))

model = SimpleNN()
print(model(torch.randn(100, 10)).shape)      # torch.Size([100, 1])
```

4. Line-by-line walkthrough

The 2024 `Animal` rules written as a Python class. Rules: name at least 3 characters; size small/medium/large; comfort range `low <= high`, both 0 to 50; bad values raise `ValueError` and change nothing; two animals are equal if name and size match.

```
class Animal:
    SIZES = ["small", "medium", "large"]   # class attribute: legal sizes

    def __init__(self, name, size, low, high):
        self.name = name                   # triggers the name setter below
        self.size = size                   # triggers the size setter
        self.set_comfort_range(low, high)  # checks low/high TOGETHER

    @property                              # makes animal.name a "read"
    def name(self):
        return self._name                  # data lives in _name (private-ish)

    @name.setter                           # makes animal.name = x a "write"
    def name(self, value):
        if not isinstance(value, str) or len(value) < 3:   # the rule
            raise ValueError("Name must be at least 3 characters.")
        self._name = value                 # reached only when rule passed

    @property
    def size(self):
        return self._size

    @size.setter
    def size(self, value):
        if value not in Animal.SIZES:      # 'in' on a list of allowed values
            raise ValueError(f"Size must be one of {Animal.SIZES}.")
        self._size = value

    @property
    def low(self):                         # read-only: no setter defined
        return self._low

    @property
    def high(self):
        return self._high

    def set_comfort_range(self, low, high):
        if not (0 <= low <= high <= 50):   # one chained check for both
            raise ValueError("Need 0 <= low <= high <= 50.")
        self._low = low                    # assign only AFTER the check
        self._high = high

    def __str__(self):                     # what print(animal) shows
        return (f"{self.name} ({self.size}, "
                f"comfortable {self.low}-{self.high} C)")

    def __eq__(self, other):               # Java equals(): name AND size
        return (isinstance(other, Animal)
                and self.name == other.name and self.size == other.size)

    def __hash__(self):                    # needed once __eq__ is defined
        return hash((self.name, self.size))

rex = Animal("Rex", "large", 5, 30)
print(rex)                                 # Rex (large, comfortable 5-30 C)
print(rex == Animal("Rex", "large", 0, 10))      # True (range ignored)
for bad in [("Al", "small", 0, 10), ("Kit", "tiny", 0, 10),
            ("Kit", "small", 20, 10), ("Kit", "small", 0, 60)]:
    try:
        Animal(*bad)                       # * unpacks the tuple to arguments
    except ValueError as error:
        print("Rejected:", error)
try:
```

```
    rex.set_comfort_range(40, 10)          # update with bad values...
except ValueError:
    print(rex.low, rex.high)               # ...unchanged: 5 30
```

How the "change should not take effect" rule is met: every `raise` comes BEFORE the line that stores the value. When `Animal("Al", ...)` fails, the object is never created at all; when `set_comfort_range(40, 10)` fails, `_low` and `_high` still hold the old numbers.

Why `_name` and not `name` inside the setter? If the setter did `self.name = value` it would call itself forever (`RecursionError`). The underscore name is the real storage box; `name` is the guarded door in front of it.

5. Traps & debugging

Mistake	Error / symptom	Fix
Forgot `self` in `def report():`	`TypeError: report() takes 0 positional arguments but 1 was given` (newer Python writes `Employee.report()`)	`def report(self):`
Wrote `name = name` in `__init__`	no error, but the object has no `.name`: `AttributeError: 'Employee' object has no attribute 'name'`	`self.name = name`
Used `name` instead of `self.name` inside a method	`NameError: name 'name' is not defined`	`self.name`
`Employee("Som")` with missing args	`TypeError: __init__() missing 1 required positional argument: 'score'`	pass all, or give a default `active=True`
`emp.report` without brackets	prints `<bound method Employee.report ...>` and does nothing	`emp.report()`
`print(emp)` shows `<__main__.Employee object at 0x...>`	no `__str__`	add `__str__` that RETURNS a string
`__str__` that uses `print(...)`	`TypeError: __str__ returned non-string (type NoneType)`	`return f"..."`
Child `__init__` without `super().__init__(...)`	`AttributeError: 'Manager' object has no attribute 'name'`	call `super().__init__(name, score, active)` first
`super().__init__()` missing in `nn.Module` subclass	`AttributeError: cannot assign module before Module.__init__() call`	first line of `__init__`
`class Manager(Employee)` but wrong parent args	`TypeError` from the parent's `__init__`	match the parent's parameters
Setter does `self.name = value`	`RecursionError: maximum recursion depth exceeded`	store in `self._name`
Validation placed AFTER the assignment	bad value stored even though the error is raised	`raise` first, assign last
`members = []` as a class attribute	every object shares ONE list	create it in `__init__`: `self.members = []`
`def __init__(self, team=[])`	all objects share the default list	`team=None`, then `self.team = team or []`
`a == b` on two objects with no `__eq__`	`False` even when all fields match	define `__eq__` (and `__hash__`)
Defined `__eq__` but no `__hash__`	`TypeError: ... (unhashable type: 'Animal')` when used in a set / dict key	add `__hash__`
Method both prints and returns, or only prints	caller cannot reuse the result	return values; print in the loop outside
`return` inside a `for` too early	only the first item is checked	put `return` after the loop, or only on a match

Mistake	Error / symptom	Fix
Mixed tabs/spaces	`IndentationError`	4 spaces

NOTE: Error wording varies slightly by Python version (3.10+ prefixes the class name, e.g. `Employee.__init__() missing ...`). Read the LAST line of the traceback and the line number above it.

Java to Python reminders: no `new`, no type declarations, no `private` keyword (underscore is a polite "do not touch"), `null` is `None`, `this` is `self` and must be written as the first parameter, `IllegalArgumentException` becomes `ValueError`.

6. Anticipated exam questions

Q1 [4 marks] Write a class `Book` with title, author and year. Add a method `describe()` that returns `"Title by Author (Year)"`. Create two books and print their descriptions.

```
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year

    def describe(self):
        return f"{self.title} by {self.author} ({self.year})"

for b in [Book("The Signalman", "Charles Dickens", 1866),
          Book("Impossible Creatures", "Katherine Rundell", 2023)]:
    print(b.describe())
```

Q2 [3 marks] What is printed, and why?

```
class Dog:
    sound = "Woof"
    def __init__(self, name):
        self.name = name
a, b = Dog("Rex"), Dog("Fido")
Dog.sound = "Bark"
b.sound = "Yap"
print(a.sound, b.sound)
```

Answer: `Bark Yap`. `a` has no `sound` of its own, so it reads the class value (changed to Bark). `b.sound = "Yap"` created an instance attribute that hides the class one.

Q3 [5 marks] `Book` years must be 1450 to 2025 and the title must not be blank. Raise `ValueError` for bad values, on creation and on update, leaving the old value unchanged. Answer: template 3.3 (validate with a `year` setter or a `set_year()` method that checks BEFORE assigning; `__init__` calls it; `if not title.strip(): raise ValueError(...)`). Marks: raises in `__init__` (1), raises on update (1), exact rule (1), old value kept (1), caught with `try/except` in the demo (1).

Q4 [3 marks] Make `print(book)` display `The Signalman, 1866`.

```
    def __str__(self):                    # add inside class Book
        return f"{self.title}, {self.year}"
```

Q5 [4 marks] Create `Manager(Employee)` with an extra `team` list and a `lead_team()` method. Its `report()` should show the Employee line then the team size. Answer: template 3.4 (`super().__init__(...)`, `super().report()`).

Q6 [4 marks] Given the Employee list from 3.5, print each employee's eligibility, then how many are eligible. `eligible_for_training` must return True/False.

```
count = 0
for p in people:
    if p.eligible_for_training():
        print(p.name, "eligible")
        count += 1
    else:
```

```
        print(p.name, "not eligible")
print(count, "eligible")
```

Q7 [5 marks] Write class `Library` that starts empty and has `add_book(book)`, `find_by_author(author)` (returns a list), `remove_book(title)` (raise `ValueError` if absent) and `total_books()`. Answer: template 3.6, using `[b for b in self.books if b.author == author]` for the finder and `len(self.books)` for the total.

Q8 [3 marks] Find three errors.

```
class Pet:
    def __init__(name, age):
        name = name
        self.age = age
    def show():
        print(self.name)
```

Answer: (1) `__init__` is missing `self` as first parameter; (2) `name = name` should be `self.name = name`; (3) `show` is missing `self` (and its `self.name` would then be undefined).

Q9 [2 marks] `Manager` extends `Employee`. State the result of `isinstance(boss, Employee)` and `isinstance(worker, Manager)`. Answer: `True` (a Manager is an Employee); `False` (an ordinary Employee is not a Manager).

Q10 [3 marks] Two `Animal` objects are equal when name and size match. Implement it. Answer: `__eq__` and `__hash__` from section 4.

7. Quick-reference box

```
class Name:                     CamelCase; def __init__(self, a, b):
    shared = 0                  class attribute (all objects); Name.shared
    self.a = a                  instance attribute (per object)
    def method(self): ...       self is ALWAYS the first parameter
    def __str__(self): return f"..."     RETURN a str; used by print()
    def __repr__(self): ...     shown inside lists / debugging
obj = Name(1, 2); obj.method()  no new keyword; call WITH brackets
VALIDATE:  if bad: raise ValueError("msg")   -- BEFORE assigning
CALLER:    try: Name(...)  except ValueError as e: print(e)
PROPERTY:  @property def x(self): return self._x
           @x.setter def x(self, v): check...; self._x = v   (store in _x)
INHERIT:   class Child(Parent):  super().__init__(args) FIRST line
OVERRIDE:  same method name in child; super().method() to reuse parent
isinstance(obj, Class)   True for the class AND its parents
EQUALITY:  def __eq__(self, o): ...   and   def __hash__(self): hash((..))
MANAGER:   self.items = []  in __init__ (never as class attribute)
           add / find (return obj or None) / remove (ValueError) / total
methods RETURN values; the loop outside the class PRINTS them
nn.Module: class M(nn.Module): super().__init__() first; def forward(self,x)
```

Appendix (optional): the 2024 Java pet-care task as Python practice

The ideas transfer one-to-one. If you can write this, you can write any "classes + a manager" task.

Java (2024 paper)	Python
`Animal(name, size, low, high)` constructor	`def __init__(self, name, size, low, high)`
getters / setters	`@property` and `@x.setter` (or `set_x()` methods)
`throw new IllegalArgumentException`	`raise ValueError(...)`
`toString()`	`__str__`
`equals()` / `hashCode()`	`__eq__` / `__hash__`
enum `AnimalSize`	list/dict of allowed sizes: `SIZE_ORDER = {"small": 1, ...}`
`occupant = null`	`self._occupant = None`
`Enclosure`: getters, NO setters	properties with no `.setter`
`checkCompatibility(Animal)`	`check_compatibility(self, animal)` returns bool
`PetService` with `ArrayList<Enclosure>`	`PetService` with `self.enclosures = []`
`allocateAnimal`: cheapest free compatible enclosure	list comprehension + `min(..., key=...)`

```
SIZE_ORDER = {"small": 1, "medium": 2, "large": 3}   # replaces the Java enum

class Enclosure:                                # uses Animal from section 4
    def __init__(self, size, temperature, running_cost):
        self._size = size
        self._temperature = temperature
        self._running_cost = running_cost
        self._occupant = None                   # Java null

    @property
    def size(self):                             # getters only: no setters
        return self._size
    @property
    def temperature(self):
        return self._temperature
    @property
    def running_cost(self):
        return self._running_cost
    @property
    def occupant(self):
        return self._occupant

    def check_compatibility(self, animal):      # same or smaller size AND fits
        return (SIZE_ORDER[animal.size] <= SIZE_ORDER[self.size]
                and animal.low <= self.temperature <= animal.high)

    def add_animal(self, animal):
        if self._occupant is not None or not self.check_compatibility(animal):
            raise ValueError("Animal cannot be added to this enclosure.")
        self._occupant = animal

    def remove_animal(self):                    # empty enclosure: do nothing
        self._occupant = None

    def __str__(self):
        who = self._occupant.name if self._occupant else "empty"
        return (f"{self.size} enclosure, {self.temperature} C, "
                f"cost {self.running_cost}/day, {who}")

class PetService:
    def __init__(self):
        self.enclosures = []                    # initially empty

    def add_enclosure(self, enclosure):
        self.enclosures.append(enclosure)
```

```
    def print_all_enclosures(self):
        for enclosure in self.enclosures:
            print(enclosure)

    def allocate_animal(self, animal):         # True if housed, else False
        options = [e for e in self.enclosures
                   if e.occupant is None and e.check_compatibility(animal)]
        if not options:
            return False                       # no change made
        cheapest = min(options, key=lambda e: e.running_cost)
        cheapest.add_animal(animal)
        return True

    def remove_animal(self, animal):
        for enclosure in self.enclosures:
            if enclosure.occupant == animal:   # uses Animal.__eq__
                enclosure.remove_animal()
                return
        raise ValueError("Animal is not in the service.")

service = PetService()
service.add_enclosure(Enclosure("large", 20, 9))
service.add_enclosure(Enclosure("medium", 20, 5))
service.add_enclosure(Enclosure("small", 20, 2))
print(service.allocate_animal(Animal("Rex", "medium", 10, 25)))   # True
print(service.allocate_animal(Animal("Max", "large", 30, 40)))    # False
service.print_all_enclosures()                 # Rex went to the cost-5 pen
service.remove_animal(Animal("Rex", "medium", 0, 1))   # equal by name+size
```